
University of Michigan–Ann Arbor
Department of Electrical Engineering and Computer Science
EECS 475 **Introduction to Cryptography**, Winter 2024

Discussion 7: Review Guide up to Lecture 14
10/11/2024

Contents

1 Overview	3
2 Perfect Secrecy	3
2.1 Definitions/Content Review	3
2.1.1 Shannon secrecy	3
2.1.2 Perfect Secrecy	4
2.1.3 One-Time Pad	4
2.1.4 Flaws with perfect secrecy	4
2.2 Quiz Questions	4
2.3 Discussion Problems/Additional Practice	5
3 Negligible Functions	8
3.1 Definitions/Content Review	8
3.1.1 Polynomially Bounded	8
3.1.2 Negligible Function	8
3.1.3 Uses	8
3.2 Quiz Questions	8
3.3 Discussion Problems/Additional Practice	9
4 Computational Indistinguishability	9
4.1 Definitions/Content Review	9
4.1.1 Ensemble	9
4.1.2 Predictor and Distinguisher Definitions	10
4.1.3 EAV Security	10
4.1.4 Uses	10
4.2 Quiz Questions	11
5 Pseudorandom Generators	11
5.1 Definitions/Content Review	11
5.1.1 Pseudorandom Generators	11
5.1.2 Distinguishing Game	12
5.1.3 Pseudorandom One-Time-Pad	12
5.1.4 Pros and Cons of Pseudorandom One-Time-Pad and PRGs	12

5.1.5	Big Idea in Hybrid Proofs	12
5.2	Discussion Problems	13
6	Multi-message Security	15
6.1	Definitions/Content Review	15
6.1.1	Multi-Message EAV Security, Multi-Message CPA Security, and CPA Security	15
6.1.2	Uses	17
6.2	Quiz Questions	17
7	Pseudorandom Functions	18
7.1	Definitions/Content Review	18
7.1.1	Random Functions	18
7.1.2	Lazy Construction of Random Function	18
7.1.3	Definition of PRF	18
7.1.4	CPA Security with PRFs	20
7.1.5	Uses	20
7.1.6	Big Idea in Hybrid Proofs	20
7.2	Quiz Questions	20
7.3	Discussion Problems/ Additional Practice	21
8	Message Authentication	23
8.1	Definitions/Content Review	23
8.1.1	Existentially Unforgable under a Chosen-Message Attack	23
8.1.2	Strongly Unforgable under Chosen-Message Attack	23
8.1.3	PRFs Make Good MACs for Fixed Length Messages	24
8.1.4	Common Attacks on arbitrary length MACs	24
8.1.5	MACs for Arbitrary Length Messages	24
8.1.6	Uses	24
8.2	Quiz Questions	25
8.3	Discussion Problems	26
9	Hash Functions	26
9.1	Definitions/Content Review	26
9.1.1	Collision Resistance	26
9.1.2	Hash-and-Mac for Arbitrary Length Messages	26
9.1.3	Merkle Hash Trees	27
9.1.4	Uses	27
9.1.5	Big Idea for Proofs	27
9.2	Quiz Questions	27
9.3	Discussion Problems	28
10	Number Theory	29
10.1	Definitions/Content Review	29
10.1.1	Groups	29
10.1.2	Potentially Useful Theorems	30
10.2	Quiz Questions	30

10.3 Discussion Problems	31
11 Key Exchange and Number Theory Assumptions	31
11.1 Definitions/Content Review	31
11.1.1 Discrete Log Assumption	31
11.1.2 Decisional Diffie-Hellman Assumption	31
11.1.3 Key Exchange Security	32
11.1.4 Diffie-Hellman Key Exchange	32
11.1.5 Uses	33
11.2 Quiz Questions	33
12 Public-Key Cryptography	33
12.1 Definitions/Content Review	33
12.1.1 Correctness	33
12.1.2 Security	33
12.1.3 El Gamal Encryption	34
12.1.4 Uses	34
12.2 Quiz Questions	34
12.3 Discussion Problems	35

1 Overview

This document is supposed to be an overview of the topics discussed in this course up to lecture 14. We have proven all of the theorems listed in class or discussion in the past. If you are comfortable with the HW questions, discussion questions, and the proofs of theorems in this document, you should be prepared for the exam.

2 Perfect Secrecy

2.1 Definitions/Content Review

2.1.1 Shannon secrecy

A cryptosystem $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is *Shannon secret*, or *perfectly secret*, if for all probability distributions $\mathcal{D}$ over $\mathcal{M}$ and any fixed message $\bar{m} \in \mathcal{M}$ and ciphertext $\bar{c} \in \mathcal{C}$, we have:

$$\Pr_{\substack{m \leftarrow \mathcal{D} \\ k \leftarrow \text{Gen}}} [m = \bar{m} \mid \text{Enc}_k(m) = \bar{c}] = \Pr_{m \leftarrow \mathcal{D}} [m = \bar{m}]$$

In other words, the “a priori” message distribution (before seeing the ciphertext) is identical to the “a posteriori” distribution (after seeing, i.e., conditioned on the ciphertext). Learning that the ciphertext is $\bar{c}$ gives the attacker *no extra information* on what the message m might be, because each potential message has the same probability of being the “real” one as before the ciphertext was revealed.

2.1.2 Perfect Secrecy

Recall that a cryptosystem $(\text{Gen}, \text{Enc}, \text{Dec})$ with message space $\mathcal{M}$ is *Shannon secret* if for all probability distributions $\mathcal{D}$ over $\mathcal{M}$ and any fixed message $\bar{m} \in \mathcal{M}$ and ciphertext $\bar{c} \in \mathcal{C}$:

$$\Pr_{\substack{m \leftarrow \mathcal{D} \\ k \leftarrow \text{Gen}}} [m = \bar{m} \mid \text{Enc}_k(m) = \bar{c}] = \Pr_{m \leftarrow \mathcal{D}} [m = \bar{m}].$$

In words: the (prior) distribution of the message remains unchanged, even *after* we reveal (i.e., condition on) the ciphertext.

The system is *perfectly secret* if $\forall m_0, m_1 \in \mathcal{M}, \bar{c} \in \mathcal{C}$:

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}_k(m_0) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [\text{Enc}_k(m_1) = \bar{c}].$$

In words: each message, when encrypted under a random key produced by Gen , results in *exactly the same distribution of ciphertext*.

Theorem 2.1. *Perfect secrecy is equivalent to Shannon secrecy. (Lecture 3)*

Theorem 2.2. *If $(\text{Gen}, \text{Enc}, \text{Dec})$ is a perfectly secret encryption scheme with message space $\mathcal{M}$ and key space $\mathcal{K}$, then $|\mathcal{K}| \geq |\mathcal{M}|$.*

2.1.3 One-Time Pad

The one-time pad is a cryptosystem with the following specification:

$$\begin{aligned} \mathcal{M} = \mathcal{C} = \mathcal{K} &= \{0, 1\}^\ell, \text{ where } \ell \geq 1 \\ \text{Gen} : & \text{ choose } k \leftarrow \mathcal{K} \text{ uniformly at random} \\ \text{Enc}_k(m) : & \text{ output } c = m \oplus k \\ \text{Dec}_k(c) : & \text{ output } c \oplus k \end{aligned}$$

Theorem 2.3. *The one-time pad is perfectly secret.*

2.1.4 Flaws with perfect secrecy

Perfect Secrecy has two fundamental limitations: (1) if Alice encrypts two messages with the same key k , all security is lost; and (2) the key k must be uniformly random and be (at least) as long as the message.

2.2 Quiz Questions

Exercise 1

Which of the following statements is(are) true?

- (a) Shannon secrecy is strictly stronger than perfect secrecy.
- (b) Shannon secrecy is equivalent to perfect secrecy.
- (c) Shannon secrecy is strictly weaker than perfect secrecy.

Solution

(b)

Exercise 2

If we want to encrypt a message of length 12 that achieves perfect secrecy, how long could the length of the key be?

(a) 10

(b) 15

(c) 13

(d) 8

Solution

(b),(c)

2.3 Discussion Problems/Additional Practice

Exercise 3

In the one-time pad, there is some possibility that $k = 0^\ell$, in which case we have $\text{Enc}_k(m) = m$ for any $m \in \mathcal{M}$. Intuitively, it might seem like a bad idea to send the message “in the clear” like this. So, to avoid this possibility, we could modify the scheme to make $\mathcal{K} = \{0, 1\}^\ell \setminus \{0^\ell\}$. Is this modified scheme still perfectly secret?

Solution

No. If Eve intercepts the ciphertext $\bar{c} \in C$, she knows immediately that $\bar{c}$ could not have been the original message because we disallow $k = 0^\ell$. Formally:

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}_k(\bar{c}) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [k = 0^\ell] = 0.$$

Let $\bar{m} \in \mathcal{M} \setminus \{\bar{c}\}$ be arbitrary. It follows that:

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}_k(\bar{m}) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [k = \bar{m} \oplus \bar{c}] = \frac{1}{2^\ell - 1}$$

We conclude that the modified scheme is not perfectly secret because $0 \neq \frac{1}{2^\ell - 1}$.

Exercise 4

Eve knows that Alice and Bob are encrypting their messages with the one-time pad. She has been saving all of their transmitted ciphertexts, and it seems that they are using the system properly (e.g., they are not reusing any keys). Being desperate to eavesdrop on their conversation, one day Eve decides to crack one of the ciphertexts by using her ultra-powerful ∞ Pad computer to perform a brute-force search over the entire key space. Will she be able to recover the original message in this way?

Solution

No. Suppose Eve tries to crack the ciphertext $\bar{c}$ with her ∞ Pad. The set of messages she recovers is:

$$\{\bar{c} \oplus k : k \in \mathcal{K}\} = \mathcal{M}.$$

Indeed, the original message is in $\mathcal{M}$, but so is every other possible message—so she has no way of determining which message was the real one.

Exercise 5

Imagine a scheme similar to the One-Time Pad, where $\oplus$ is replaced with $\wedge$ (bitwise AND). Is this a valid encryption scheme? Is it perfectly secure?

Solution

This is not a valid encryption scheme because if $k = 0^l$ then $Enc_k(m) = 0^l$ for all $m \in \mathcal{M}$. This means decryption is impossible.

This scheme is also not perfectly secure since if any bit in the ciphertext is 1, then the corresponding bit in the plaintext must also be 1 since if it were zero then the bit would be 0 in the ciphertext. Formally, if $m = 1^l$ and $\bar{m} = 0^l$ and $\bar{c} = 1^l$ then:

$$\Pr_{k \leftarrow \text{Gen}} [Enc_k(\bar{m}) = \bar{c}] = 0 \neq \Pr_{k \leftarrow \text{Gen}} [Enc_k(m) = \bar{c}]$$

Exercise 6

How can we generalize One-Time-Pad to deal with alphabets larger than 2 characters? WLOG you may assume that an alphabet of size b is simply $\{0, 1, 2, \dots, b-1\}$ and $\mathcal{M} = \mathcal{K} = \{0, 1, 2, \dots, b-1\}^n$

Solution

Instead of using $\oplus$ we can use character wise addition modulo b (note that $\oplus$ is a special case of this). Decrypting would then be subtracting each character by the corresponding character in the key.

Exercise 7

Prove or give a counter example. In a perfectly secret encryption scheme, given the encryption $c \leftarrow \text{Enc}_k(m)$ of a *uniformly random* message $m \leftarrow \mathcal{M}$ under a key $k \leftarrow \text{Gen}$, the secret key k is uniformly random over $\mathcal{K}$. Formally: for any fixed $\bar{k} \in \mathcal{K}$ and $\bar{c} \in \mathcal{C}$,

$$\Pr_{k \leftarrow \text{Gen}, m \leftarrow \mathcal{M}} [k = \bar{k} \mid \text{Enc}_k(m) = \bar{c}] = 1/|\mathcal{K}|.$$

Solution

It is *false*.

Define $\mathcal{M} = \{0, 1\}^n$, $\mathcal{K} = \{0, 1\}^{n+1}$ and $\mathcal{C} = \{0, 1\}^{n+1}$,

$$\text{Enc}_k(m) = (m \oplus k_{[1\dots n]}) || k_{n+1}, \quad (2.1)$$

where $k_{[1\dots n]}$ means the first n bits of k , and k_{n+1} means the $(n + 1)$ th bit of k . Note that

$$((m \oplus k_{[1\dots n]}) || k_{n+1}) \oplus k = m || 0$$

and Gen chooses a string uniformly from the key space and returns.

Therefore, given the ciphertext ended with 1; i.e. $\bar{c}_{n+1} = 1$, we know that the $k_{n+1} = 1$, so the key is not uniformly distributed. This scheme is perfectly secret, because for any $m_0 \in \mathcal{M}$,

$$\begin{aligned} \Pr_{k \leftarrow \text{Gen}} [\text{Enc}(m_0) = \bar{c}] &= \Pr_{k \leftarrow \text{Gen}} [m_0 \oplus k_{[1\dots n]} || k_{n+1} = \bar{c}] \\ &= \Pr_{k \leftarrow \text{Gen}} [m_0 \oplus k_{[1\dots n]} = \bar{c}_{[1\dots n]} \wedge k_{n+1} = \bar{c}_{n+1}]. \\ &= \Pr_{k \leftarrow \text{Gen}} [k_{[1\dots n]} = m_0 \oplus \bar{c}_{[1\dots n]} \wedge k_{n+1} = \bar{c}_{n+1}]. \\ &= \frac{1}{2^n} \times \frac{1}{2} = \frac{1}{2^{n+1}}, \end{aligned} \quad (2.2)$$

is a constant, so for any $m_0, m_1 \in \mathcal{M}$,

$$\Pr_{k \leftarrow \text{Gen}} [\text{Enc}(m_0) = \bar{c}] = \Pr_{k \leftarrow \text{Gen}} [\text{Enc}(m_1) = \bar{c}]. \quad (2.3)$$

3 Negligible Functions

3.1 Definitions/Content Review

3.1.1 Polynomially Bounded

A non-negative function $f: \mathbb{N} \rightarrow \mathbb{R}$ is *polynomially bounded*, written $f(n) = \text{poly}(n)$, if $f(n) = O(n^c)$ for some constant $c \geq 0$.

3.1.2 Negligible Function

A non-negative function $\varepsilon: \mathbb{N} \rightarrow \mathbb{R}$ is *negligible*, written $\varepsilon(n) = \text{negl}(n)$, if it decreases faster than the inverse of any polynomial. Formally: $\lim_{n \rightarrow \infty} \varepsilon(n) \cdot n^c = 0$ for any constant $c \geq 0$. (Otherwise, we say that $\varepsilon(n)$ is *non-negligible*.)

Theorem 3.1. *A polynomially bounded function multiplied by a negligible function is a negligible function.*

Theorem 3.2. *Polynomially bounded functions are closed under addition and multiplication*

Theorem 3.3. *Negligible functions are closed under addition and multiplication*

3.1.3 Uses

Negligible functions are used to define security definitions going forward.

3.2 Quiz Questions

Exercise 8

Suppose $p_1(n)$ and $p_2(n)$ are two polynomial functions. Choose which of the following **MUST** be polynomials?

- (a) $p_1(n)^{p_2(n)}$
- (b) $p_1(n) + p_2(n)$
- (c) $p_1(n) \cdot p_2(n)$

Solution

(b),(c)

Exercise 9

Suppose $\text{negl}_1(n)$ and $\text{negl}_2(n)$ are two negligible functions in n , and $p(n)$ is a polynomial function. Which of the following MUST be negligible?

- (a) $\text{negl}_1(n) \cdot p(n)$
- (b) $\text{negl}_1(n) + \text{negl}_2(n)$
- (c) $\text{negl}_1(n)/\text{negl}_2(n)$
- (d) $(\text{negl}_1(n))^{p(n)}$

Solution

(a),(b),(d)

3.3 Discussion Problems/Additional Practice

Exercise 10

Which of the following are negligible?

- (a) $\frac{1}{2^{\log n}}$
- (b) $\frac{1}{n^{\log n}}$
- (c) $\frac{1}{(\log n)^{\log n}}$

Solution

(b),(c)

4 Computational Indistinguishability

4.1 Definitions/Content Review

4.1.1 Ensemble

A sequence $\{X_n\}_{n \in \mathbb{N}}$ is an ensemble, if for any $n \in \mathbb{N}$, X_n is a distribution over $\{0, 1\}^*$.

4.1.2 Predictor and Distinguisher Definitions

Given two ensembles, $\{X_n^0\}_{n \in \mathbb{N}}, \{X_n^1\}_{n \in \mathbb{N}}$ where X_n^0 and X_n^1 are distributions over $l(n)$ for some polynomial $l(\cdot)$, we say X_n^0 and X_n^1 are computationally indistinguishable if $\forall \text{PPT}, \mathcal{A}, \exists \text{negl}(n)$ such that for all n ,

Definition 4.1 (Predictor).

$$\Pr_{b \leftarrow \{0,1\}, x \leftarrow X_n^b} [\mathcal{A}(x) = b] \leq \frac{1}{2} + \text{negl}(n)$$

Definition 4.2 (Distinguisher).

$$\Pr_{x_0 \leftarrow X_n^0} [\mathcal{A}(x_0) = 1] - \Pr_{x_1 \leftarrow X_n^1} [\mathcal{A}(x_1) = 1] \leq \text{negl}(n)$$

Theorem 4.3. *The predictor and distinguisher definitions are equivalent*

Theorem 4.4 (Informal Closure Lemma). *Computationally indistinguishable ensembles are closed under efficient operations.*

Theorem 4.5 (Informal Hybrid Lemma). *Computational indistinguishability is a transitive relation up to a polynomial amount.*

4.1.3 EAV Security

Experiment $\text{Expt}_{\text{EAV}}^{b, \mathcal{A}}(1^n)$

- $(m_0, m_1) \leftarrow \mathcal{A}(1^n)$ where $|m_0| = |m_1|$.
(Here the adversary is choosing two messages for the challenger to encrypt.)
- $k \xleftarrow{\$} \text{Gen}(1^n)$ and $c \leftarrow \text{Enc}_k(m_b)$.
(The challenger chooses one of the two messages depending on the parameter b to encrypt and returns it to the adversary.)
- $b' \leftarrow \mathcal{A}(c)$.
(The adversary outputs its guess)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is EAV-secure if and only if for any PPT $\mathcal{A}$, (we will denote computationally indistinguishable by the symbol $\approx$ here and from now on)

$$\{\text{Expt}_{\text{EAV}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{EAV}}^{1, \mathcal{A}}(1^n)\}_n.$$

4.1.4 Uses

We use computational indistinguishability in a similar way to define many of the security definitions going forward. This allows us to get rid of the restrictions of perfect security.

4.2 Quiz Questions

Exercise 11

Suppose we have a sequence of distributions $X_1, \dots, X_m$ such that X_i is computationally indistinguishable from X_{i+1} for i in $1, \dots, m-1$. Are X_1 and X_m computationally distinguishable?

Solution

It depends on the length of m , if m is polynomial, then yes by hybrid lemma, but if m is super polynomial, then perhaps not.

Exercise 12

Suppose X and Y are two computationally indistinguishable distributions. Let M be a function that takes an output from the distributions as an input. Define $M(X)$ as the distribution of $M(x)$ s.t. $x \leftarrow X$. Are $M(X)$ and $M(Y)$ computationally indistinguishable?

Solution

It depends on the runtime of M , if M is a PPT, then yes by closure, but if M runs in super polynomial time, then perhaps not.

5 Pseudorandom Generators

5.1 Definitions/Content Review

5.1.1 Pseudorandom Generators

A deterministic, polynomial-time algorithm G is a *pseudorandom generator*, abbreviated PRG, if the following two properties hold:

1. *expansion*: G maps each n -bit string to an $\ell(n)$ -bit string, where $\ell(n) > n$ (for all n) is called the *expansion* of G ; i.e. $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ where the output is longer than the input, and
2. *pseudorandomness*: for a uniformly random “seed” string $s \leftarrow \{0, 1\}^n$, $G(s)$ is indistinguishable from a uniformly random $\ell(n)$ -bit string, as formalized by the distinguishing game below.

Looking ahead, PRGs will let us circumvent the “ $|\mathcal{K}| \geq |\mathcal{M}|$ ” requirement from our treatment of Shannon secrecy, while still achieving computational security.

5.1.2 Distinguishing Game

Let G be a candidate PRG with expansion $\ell(n) > n$. A *distinguisher* D is a (possibly randomized) algorithm that, given the public security parameter n and an $\ell(n)$ -bit string y as input, outputs a bit indicating its “decision.” By convention, an output of 1 (“accept”) means that D is declaring its judgement that y is pseudorandom, while an output of 0 (“reject”) indicates its judgement that y is truly random. The first input is a dummy string, usually 1^n by convention, which makes explicit the fact that the distinguisher knows n and is allowed to run in $\text{poly}(n)$ time.

The *advantage* of a distinguisher D against G is the difference between the probability that D accepts G ’s output on a random seed (true positive rate) and the probability that D accepts a truly random string (false positive rate). Formally:

$$\mathbf{Adv}_G(D) \triangleq \left| \Pr_{s \leftarrow \{0,1\}^n} [D(1^n, G(s)) = 1] - \Pr_{y \leftarrow \{0,1\}^{\ell(n)}} [D(1^n, y) = 1] \right|.$$

Notice that the advantage is a function of the security parameter n : each value of n defines a different “experiment” and difference of probabilities.

Definition: G is *pseudorandom* if for every (randomized) polynomial-time distinguisher D , its advantage against G is negligible: $\mathbf{Adv}_G(D) = \text{negl}(n)$. In other words, G “fools” every feasible distinguisher.

Theorem 5.1. *If a PRG exists, there is a length doubling PRG.*

5.1.3 Pseudorandom One-Time-Pad

The one-time-pad with a pseudorandom pad is a cryptosystem with the following specification where $G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ is a PRG:

$\mathcal{M} = \mathcal{C} = \{0, 1\}^{\ell(n)}, \mathcal{K} = \{0, 1\}^n$, where $\ell = \text{poly}(n)$

Gen : choose $k \leftarrow \mathcal{K}$ uniformly at random

Enc $_k(m)$: output $c = m \oplus G(k)$

Dec $_k(c)$: output $c \oplus G(k)$

Theorem 5.2. *The one-time pad with a pseudorandom pad is EAV-secure.*

Proof. The proof sketch is that by pseudorandomness, the distributions in each case, Enc $_k(m_1)$ or Enc $_k(m_0)$ are indistinguishable from a random string distribution. Thus by the hybrid lemma, they are indistinguishable from each other.

5.1.4 Pros and Cons of Pseudorandom One-Time-Pad and PRGs

PRG’s allow us to use a much shorter key to ensure a meaningful amount of security. However, it does not solve the problem of key reuse.

5.1.5 Big Idea in Hybrid Proofs

You can make hybrids where you replace pseudorandom outputs with random strings. These have to be computationally indistinguishable from the original distribution.

5.2 Discussion Problems

Exercise 13

For each of the following functions G , describe a feasible distinguisher D and show that it has a non-negligible advantage against G .

- (a) $G(s) = s \| b$, where $b = \bigoplus_{i=1}^n s_i$ is the $\oplus$ of all the bits of s .
- (b) $G(s) = s \| b$, where $b = s_1 \wedge s_2$. (first bit of s AND second bit of s).
- (c) $G(s) = s \| s'$, where $s' = (as + c) \bmod m$ for some fixed parameters a, c, m .

Solution

- (a) We propose the following distinguisher:

$$D(y \in \{0, 1\}^{n+1}) : \text{if } y_{n+1} = \bigoplus_{i=1}^n y_i, \text{ accept; otherwise, reject.}$$

We claim that D has a non-negligible advantage against G . By construction, D will accept every output of G , so $\Pr[D(1^n, G(s)) = 1] = 1$ for all $s \in \{0, 1\}^n$. If $y \leftarrow \{0, 1\}^{n+1}$ is truly random, then y_{n+1} and $z := \bigoplus_{i=1}^n y_i$ are both uniformly random over $\{0, 1\}$ and independent, so $\Pr[D(1^n, y) = 1] = \Pr[y_{n+1} = 0] \cdot \Pr[z = 0] + \Pr[y_{n+1} = 1] \cdot \Pr[z = 1] = \frac{1}{2}$. Therefore, $\text{Adv}_G(D) = 1 - \frac{1}{2} = \frac{1}{2} \neq \text{negl}(n)$.

- (b) For random s , the final bit of the output $G(s)$ is noticeably biased toward zero. This suggests the following distinguisher:

$$D(y \in \{0, 1\}^{n+1}) : \text{if } y_{n+1} = 0, \text{ accept; otherwise, reject.}$$

For a truly random seed $s \leftarrow \{0, 1\}^n$, we have $\Pr[D(1^n, G(s)) = 1] = \Pr[s_1 \wedge s_2 = 0] = 1 - \Pr[s_1 = 1] \cdot \Pr[s_2 = 1] = \frac{3}{4}$. If $y \leftarrow \{0, 1\}^{n+1}$ is truly random, then $\Pr[D(1^n, y) = 1] = \Pr[y_{n+1} = 0] = \frac{1}{2}$. Therefore, $\text{Adv}_G(D) = \frac{3}{4} - \frac{1}{2} = \frac{1}{4} \neq \text{negl}(n)$. (There is another distinguisher that works more like the one from the previous part, and has advantage $1/2$. Either one suffices to demonstrate that G is not a PRG.)

- (c) Observe that the first half of $G(s)$ determines the last half in a way that we can check for. This suggests the following distinguisher:

$$D(y \in \{0, 1\}^{2n}) : \text{let } z_0 = y[1..n] \text{ and } z_1 = y[n+1..2n]; \\ \text{if } z_1 = (az_0 + c) \bmod m, \text{ accept; otherwise, reject.}$$

By construction, $\Pr[D(1^n, G(s)) = 1] = 1$ for all $s \in \{0, 1\}^n$. If $y \leftarrow \{0, 1\}^{2n}$ is truly random, then z_0 and z_1 are independent, so for $n \gg \log m$, $\Pr[D(1^n, y) = 1] = \frac{1}{2^n}$. Therefore, $\text{Adv}_G(D) = 1 - \frac{1}{2^n} \neq \text{negl}(n)$.

Exercise 14

Suppose that G is a PRG; from this we define another function $G'(s) := \overline{G(s)}$, the bitwise complement of $G(s)$. Is G' *necessarily* a PRG, or not? To approach this question, consider the following: if there was a feasible attack that could break G' , what would that imply about G ?

Solution

By contraposition, assume that there is a distinguisher D' for G' . We use D' to construct a distinguisher D for G . We describe below:

1. On input y , D calculates $\bar{y}$
2. input $\bar{y}$ into to D' .
3. output whatever D' outputs.

The advantage of D is equivalent to the advantage of D' . In the case that y is random, inverting still inputs a random string into D' . In the case that $y = G(s)$, we input $G'(s)$ exactly with s drawn uniformly at random from the domain. Thus the two worlds for D map directly onto the two worlds of D' , meaning they have the same advantage, so G is not a PRG.

Exercise 15

Suppose that $G : \{0, 1\}^n \rightarrow \{0, 1\}^{2n}$ is a PRG; from this we define another function, $G' : \{0, 1\}^{n+1} \rightarrow \{0, 1\}^{2n}$ as $G'(b||s) := G(s)$ where b is a variable that represents the first bit of the seed. Is G' *necessarily* a PRG, or not?

Solution

By contraposition, assume that there is a distinguisher D' for G' . We use D' to construct a distinguisher D for G . We describe below:

1. On input y , input y into D'
2. output whatever D' outputs.

The advantage of D is equivalent to the advantage of D' . In the case that y is random, the input is random to D' . In the case that $y = G(s)$, we input the distribution of G' exactly. Because the first bit of the input to G' does not matter, the output distribution is identical to G . Thus the two worlds for D map directly onto the two worlds of D' , meaning they have the same advantage, so G is not a PRG.

Exercise 16

Let G_1 and G_2 be two functions $\{0, 1\}^n \rightarrow \{0, 1\}^{2n+1}$. One of which is a PRG.

(a) Is $G' : \{0, 1\}^n \rightarrow \{0, 1\}^{2n+1} = G_1(s) \oplus G_2(s)$ necessarily a PRG?

(b) How can we construct a PRG $G : \{0, 1\}^{2n} \rightarrow \{0, 1\}^{2n+1}$

Solution

- (a) No. Consider the case where $G_1(s) = G_2(s)$. This will always produce a string of 0^{2n} . If we want, we can arbitrarily change the first bit of $G_1(s)$'s output to be 1 regardless of input so that only one of the functions is a PRG. This still would not be a PRG as it would all 0's always except maybe the first bit.
- (b) Consider $G(s_1 || s_2) = G_1(s_1) \oplus G_2(s_2)$. By contraposition, suppose that there is a distinguisher D for G . I will show a reduction to G_1 but you can make the same reduction to G_2 , meaning neither can be a PRG. Define D_1 below:

1. On input y , sample $x \leftarrow \{0, 1\}^n$ and calculate $G_2(x) \oplus y$
2. Input $G_2(x) \oplus y$ into D
3. Output what D outputs.

The advantage of D_1 is equivalent to the advantage of D . In the case that y is random, the input is random to D' since $\oplus$ with a random string produces a random string. In the case that $y = G_1(s)$, we input the distribution of G exactly as the second half of a random bit string is sampled independently from the first half. Thus the two worlds for D map directly onto the two worlds of D' , meaning they have the same advantage, so G_1 is not a PRG.

6 Multi-message Security

6.1 Definitions/Content Review

6.1.1 Multi-Message EAV Security, Multi-Message CPA Security, and CPA Security

Consider the following experiments parametrized with $b \in \{0, 1\}$:

Experiment $\text{Expt}_{\text{Meav}}^{b, \mathcal{A}}(1^n)$

- $(m_{0,1}, \dots, m_{0,t}), (m_{1,1}, \dots, m_{1,t}) \leftarrow \mathcal{A}(1^n)$ where $\forall i \in [t], |m_{0,i}| = |m_{1,i}|$.
(Here the adversary is choosing two vectors of messages for the challenger to encrypt.)
- $k \xleftarrow{\$} \text{Gen}(1^n)$ and $\forall i \in [t], c_i \leftarrow \text{Enc}_k(m_{b,i})$.
(The challenger chooses one of the two vectors depending on the parameter b to encrypt)

and returns it to the adversary.)

- $b' \leftarrow \mathcal{A}(c_1, \dots, c_t)$.
(The adversary outputs its guess)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is multi-message EAV-secure if and only if for any PPT $\mathcal{A}$,

$$\{\text{Expt}_{\text{Meav}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{Meav}}^{1, \mathcal{A}}(1^n)\}_n.$$

Experiment $\text{Expt}_{\text{LRcpa}}^{b, \mathcal{A}}(1^n)$

- $k \xleftarrow{\$} \text{Gen}(1^n)$.
- $b' \leftarrow \mathcal{A}^{\text{LR}_k^b(\cdot, \cdot)}(1^n)$ where $\text{LR}_k^b(m_0, m_1)$ outputs $\text{Enc}_k(m_b)$.
(The adversary gets oracle access to the LR_k^b oracle which will always return the encryption of the left message or right message depending on the parameter b when given a message pair. The adversary can make $\text{poly}(n)$ queries.)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is multi-message CPA-secure if and only if for any PPT $\mathcal{A}$,

$$\{\text{Expt}_{\text{LRcpa}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{LRcpa}}^{1, \mathcal{A}}(1^n)\}_n.$$

Experiment $\text{Expt}_{\text{cpa}}^{b, \mathcal{A}}(1^n)$

- $k \xleftarrow{\$} \text{Gen}(1^n)$.
- $b' \leftarrow \mathcal{A}^{\text{Enc}_k(\cdot)}(1^n)$ where $\text{Enc}_k(m)$ outputs $\text{Enc}(k, m)$.
(The adversary has access to the $\text{Enc}_k(\cdot)$ oracle which will encrypt any messages sent by $\mathcal{A}$. The adversary can make $\text{poly}(n)$ queries.)
- $(m_0, m_1) \leftarrow \mathcal{A}(1^n)$ where $|m_0| = |m_1|$.
(Here the adversary is choosing two messages for the challenger to encrypt.)
- $c \leftarrow \text{Enc}_k(m_b)$.
(The challenger chooses one of the two messages depending on the parameter b to encrypt and returns it to the adversary.)
- $b' \leftarrow \mathcal{A}(c)$.
(The adversary outputs its guess)
- Output b' .

We say that $(\text{Gen}, \text{Enc}, \text{Dec})$ is CPA-secure if and only if for any PPT $\mathcal{A}$,

$$\{\text{Expt}_{\text{cpa}}^{0, \mathcal{A}}(1^n)\}_n \approx \{\text{Expt}_{\text{cpa}}^{1, \mathcal{A}}(1^n)\}_n.$$

Theorem 6.1. *Multi-message CPA security implies multi-message EAV security*

Theorem 6.2. *Multi-message CPA security is equivalent to CPA security.*

Theorem 6.3. *In order for a scheme to be CPA secure, it must have randomized encryption.*

6.1.2 Uses

In reality, we would want to reuse keys, so these definitions capture that desire. However, multi-message EAV security does not allow for manipulation of what encryptions an adversary gets based on previous queries. In real life, adversaries may adaptively influence the encryptions they see, so we want to be secure against this behavior. CPA and multi-message CPA capture this idea.

6.2 Quiz Questions

Exercise 17

Which of the following are true?

- (a) EAV-Security implies Multi-Message EAV-Security
- (b) Multi-Message EAV-Security implies EAV-Security
- (c) Multi-Message EAV-Security is equivalent to EAV-Security

Solution

(b)

Exercise 18

Which of the following are true?

1. CPA-Security implies Multi-Message CPA-Security
2. Multi-Message CPA-Security implies CPA-Security
3. Multi-Message CPA-Security is Strictly Stronger than CPA-Security

Solution

(a),(b)

Exercise 19

Is CPA-Security stronger than multi-message EAV-Security?

Solution

Yes

7 Pseudorandom Functions

7.1 Definitions/Content Review

7.1.1 Random Functions

A function $\text{Func}_n : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a *random function* (or more pedantically, a *randomly chosen function*) if $\text{Func}_n(x)$ is a uniformly random; independent string, for each $x \in \{0, 1\}^n$. That is:

$$\Pr_{y \leftarrow \{0, 1\}^n} [\text{Func}_n(x) = y] = 1/2^n.$$

To be more precise, we can think of Func_n as a function chosen from the uniform distribution over all possible length-preserving functions. We stress that, while Func_n is chosen randomly, it truly is a *function* in the mathematical sense: whenever Func_n is evaluated on a particular input $x \in \{0, 1\}^n$, it always outputs the *same* string; i.e., the value of $\text{Func}_n(x)$ *does not change* from one call to the next.

7.1.2 Lazy Construction of Random Function

One way of choosing a random function Func_n is by precomputing a static lookup table, where entry $x \in \{0, 1\}^n$ of the table contains a uniformly random; independent string $y_x = \text{Func}_n(x)$. This process reflects the definition given above, but is completely infeasible in reality: we cannot afford to generate and write down 2^n random strings in a table, and in any case; a feasible user of Func_n would only be able to use $\text{poly}(n)$ -many entries of the table anyway.

A different, feasible way of choosing a random function is *lazily*, by maintaining a dynamically growing lookup table. To evaluate Func_n on an input $x \in \{0, 1\}^n$, we check if there is already an entry (x, y_x) in the lookup table, and output y_x if so. Otherwise, we choose $y_x \leftarrow \{0, 1\}^n$ uniformly at random, add (x, y_x) to the lookup table, and output y_x .

7.1.3 Definition of PRF

Let $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ be a *keyed function*, i.e., a *deterministic polynomial-time algorithm* that takes a key $k \in \mathcal{K}$ and input $x \in \mathcal{X}$, and returns an output $y \in \mathcal{Y}$. (To be formal, $\mathcal{K}$, $\mathcal{X}$, and $\mathcal{Y}$ all can depend on the security parameter n , but we suppress this dependence for notational simplicity.) For example, we may take $\mathcal{K} = \mathcal{X} = \mathcal{Y} = \{0, 1\}^n$, though other choices make sense as well. In essence, the **PRF game** is to distinguish between:

- (“real” world) the function $F_k(\cdot)$ for a random key $k \leftarrow \mathcal{K}$

- (“ideal” world) a uniformly random function $\text{Func}_n(\cdot)$

given *oracle* access to one or the other. The former case is often called the “real” world, while the latter is called the “ideal” world. This is because typically $F_k(\cdot)$ is used in a real cryptosystem, where we want it to “act like” its ideal counterpart of a random function.

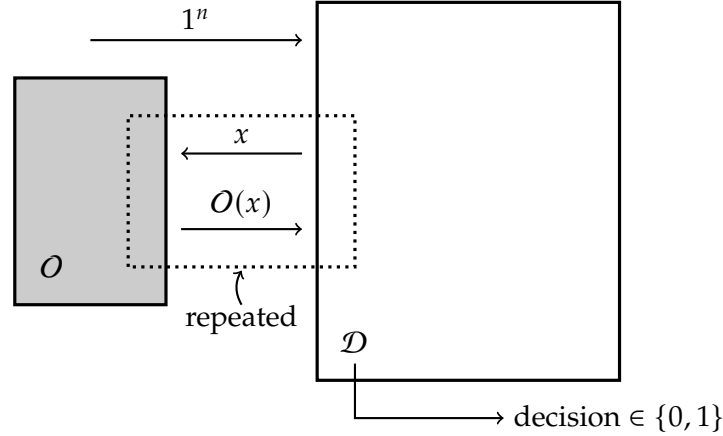
Formally, the PRF game involving a distinguisher $\mathcal{D}$ against F proceeds as follows:

1. $\mathcal{D}$ gets the security parameter 1^n as input.
2. $\mathcal{D}$ is given access to an oracle $\mathcal{O}$, which is either:
 - (a) $\mathcal{O} = F_k$, for a secret random $k \leftarrow \mathcal{K}$ that is chosen once for the entire game, or
 - (b) $\mathcal{O} = \text{Func}_n$, a uniformly random function from $\mathcal{X}$ to $\mathcal{Y}$.

The distinguisher $\mathcal{D}$ may query $\mathcal{O}$ as many times as it likes (though each query costs $\mathcal{D}$ some running time); whenever $\mathcal{D}$ queries $\mathcal{O}$ on an input x , it receives $\mathcal{O}(x)$ in response.

3. $\mathcal{D}$ outputs a decision bit.

We can represent the PRF game visually as follows:



Formally, a keyed function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ is a *pseudorandom function* (PRF) if every PPT distinguisher $\mathcal{D}$ has only negligible advantage in the PRF game against F :

$$\text{Adv}_F^{\text{prf}}(\mathcal{D}) \triangleq \left| \Pr_{k \leftarrow \mathcal{K}} [\mathcal{D}^{F_k(\cdot)}(1^n) = 1] - \Pr_{\text{random Func}_n} [\mathcal{D}^{\text{Func}_n(\cdot)}(1^n) = 1] \right| \leq \text{negl}(n).$$

Theorem 7.1. *PRG's exist if and only if PRF's exist.*

Proof. The proof sketch goes something like this:

One can construct a PRG from a PRF by just taking the first few rows and concatenating them. This should look random.

If PRG's exist, by Theorem 5.1, a length doubling PRG exists. One can create a large amount

of pseudorandom strings of the same length by recursively taking $G(L)$ and $G(R)$ continuously where L and R are the left and right sides of the previous output. The "leaves" of this recursion can be used as the outputs of the PRF with the input being used to navigate the recursion tree. You can argue this is a PRF using hybrids for each layer of the recursion tree to the next. Note that each layer gets exponentially large, so you need to take a "lazy" construction approach when querying the constructed PRF. See discussion 3 for an illustration.

7.1.4 CPA Security with PRFs

We construct the following encryption scheme where $F_k : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$ is a PRF:

$\mathcal{M} = \mathcal{C} = \{0, 1\}^{\ell(n)}, \mathcal{K} = \{0, 1\}^n$, where $\ell(n) = \text{poly}(n)$

Gen : choose $k \leftarrow \mathcal{K}$ uniformly at random

Enc $_k(m)$: $r \leftarrow \{0, 1\}^n$ uniformly at random. Output $c = (r, m \oplus F_k(r))$

Dec $_k(c)$: output $m \oplus F_k(r) \oplus F_k(r)$

Theorem 7.2. *The above construction is CPA-secure*

Proof. The proof sketch is similar to EAV security. By pseudorandomness, the distributions in each case, Enc $_k(m_1)$ or Enc $_k(m_0)$, looks indistinguishable from using a random function instead of a PRF as they are only querying a polynomial amount of times from the oracle. Thus by the hybrid lemma, they are indistinguishable from each other.

7.1.5 Uses

PRF's can be used to extend keys and to add additional randomness to a scheme to satisfy CPA security. They can also be used for MACs.

7.1.6 Big Idea in Hybrid Proofs

Replace PRF with a random function. By definition, this hybrid is computationally indistinguishable.

7.2 Quiz Questions

Exercise 20

Let $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a PRF family. How many functions are in this family?

Solution

2^n

Exercise 21

Let F be a PRF. If a PPT (polynomial probabilistic time) adversary, A , had the key, k , would A be able to distinguish F from a random function with non-negligible probability?

Solution

Yes

7.3 Discussion Problems/Additional Practice

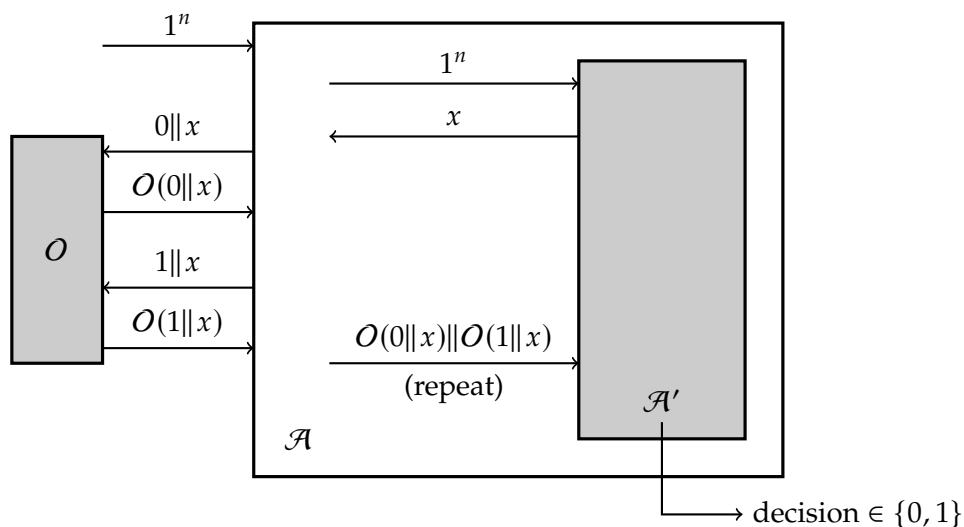
Exercise 22

Let F be a PRF and define $F'_k(x) \triangleq F_k(0\|x) \| F_k(1\|x)$. (The input length of F' is therefore one bit shorter than that of F ; e.g., if F has domain $\mathcal{X} = \{0,1\}^n$ then F' has domain $\{0,1\}^{n-1}$. If $F : \{0,1\}^n \rightarrow \{0,1\}^m$, then $F' : \{0,1\}^{n-1} \rightarrow \{0,1\}^{2m}$). We prove that F' is also a PRF, by reduction.

Remark: At a first glance, one should expect this to be a PRF, as $F_k(0\|x)$ and $F_k(1\|x)$ are seemingly random (under the assumption that F_k is a PRF). From this, any correlation between the two resulting strings should not be noticeable by an adversary.

Solution

Let $\mathcal{A}'$ be an arbitrary PPT attacker that against F' ; we need to show that $\text{Adv}_{F'}(\mathcal{A}') = \text{negl}(n)$. To do this, we use $\mathcal{A}'$ to build an attacker (a reduction) $\mathcal{A}$ against F , which works as indicated by following diagram:



Formally, $\mathcal{A}$ works as follows: when $\mathcal{A}$ receives the security parameter 1^n it passes the same parameter to $\mathcal{A}'$. Then $\mathcal{A}$ simulates an oracle $\mathcal{O}'$ to $\mathcal{A}'$ using its own oracle $\mathcal{O}$, as follows: whenever $\mathcal{A}'$ makes a query x , the reduction $\mathcal{A}$ makes two queries to $\mathcal{O}$: one for

$0\|x$ and one for $1\|x$. In response, it receives two values $O(0\|x)$ and $O(1\|x)$. It concatenates these two values and passes the result $O(0\|x)\|O(1\|x)$ to $\mathcal{A}'$, as the response to the query x . (This process is repeated for every query that $\mathcal{A}'$ makes.) When $\mathcal{A}'$ finally outputs a decision $b \in \{0, 1\}$, the reduction $\mathcal{A}$ outputs the same decision.

We now analyze the reduction $\mathcal{A}$. We consider the two worlds that it could be in. In the “real” world where $O = F_k$ for a secret random $k \leftarrow \mathcal{K}$, observe that $\mathcal{A}$ perfectly simulates an oracle $O' = F'_k$ to $\mathcal{A}'$, because it answers every query x by

$$O(0\|x)\|O(1\|x) = F_k(0\|x)\|F_k(1\|x) = F'_k(x).$$

Now consider the “ideal” world where $O = \text{Func}_n$ is a random function. Observe that $\mathcal{A}$ perfectly simulates the function

$$\text{Func}'_n(x) := \text{Func}_n(0\|x)\|\text{Func}_n(1\|x)$$

to $\mathcal{A}'$. We leave it to you to prove that Func'_n is a random function, over the random choice of Func_n alone.

It follows that

$$\begin{aligned} \text{Adv}_F^{\text{prf}}(\mathcal{A}) &= \left| \Pr_{k \leftarrow \mathcal{K}} [\mathcal{A}^{F_k(\cdot)}(1^n) = 1] - \Pr_{\text{random Func}_n} [\mathcal{A}^{\text{Func}_n(\cdot)}(1^n) = 1] \right| \\ &\stackrel{\#}{=} \left| \Pr_{k \leftarrow \mathcal{K}} [\mathcal{A}^{F'_k(\cdot)}(1^n) = 1] - \Pr_{\text{random Func}'_n} [\mathcal{A}^{\text{Func}'_n(\cdot)}(1^n) = 1] \right| \\ &= \text{Adv}_{F'}^{\text{prf}}(\mathcal{A}') \end{aligned}$$

where $\#$ follows from the fact that $\mathcal{A}$ perfectly simulates an oracle O' to $\mathcal{A}'$.

Because F is a PRF and $\mathcal{A}$ is PPT (because $\mathcal{A}'$ is), the left-most quantity is $\text{negl}(n)$, hence so is the right-most one, as desired.

Exercise 23

Is the following a secure encryption? Let f be a prf and r and uniform random bitstring. The encryption is described below:

$$\text{Enc}_k(m) = r \| f_k(m) \oplus r$$

Solution

This isn't even a correct encryption scheme. There is no way to get the message from the ciphertext.

8 Message Authentication

8.1 Definitions/Content Review

8.1.1 Existentially Unforgeable under a Chosen-Message Attack

A MAC scheme $(\text{Gen}, \text{Mac}, \text{Vrfy})$ is *existentially unforgeable under a chosen-message attack* if and only if for every PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $n \in \mathbb{N}$, $\Pr[\text{Expt}_{\text{EU}}^{\mathcal{A}}(1^n) = 1] \leq \text{negl}(n)$.

$\text{Expt}_{\text{EU}}^{\mathcal{A}}(1^n)$.

- $k \xleftarrow{\$} \text{Gen}(1^n)$;
- $(m, t) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$;
- Output 1 if $\text{Vrfy}_k(m, t) = 1$ and m not queried by $\mathcal{A}$.

In words, the adversary is able to make queries to get tags for a polynomial amount of messages and then make a message tag pair itself that is not of a message previously queried.

8.1.2 Strongly Unforgeable under Chosen-Message Attack

A MAC scheme $(\text{Gen}, \text{Mac}, \text{Vrfy})$ is *strongly existentially unforgeable under a chosen-message attack* if and only if for every PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $n \in \mathbb{N}$, $\Pr[\text{Expt}_{\text{SEU}}^{\mathcal{A}}(1^n) = 1] \leq \text{negl}(n)$.

$\text{Expt}_{\text{SEU}}^{\mathcal{A}}(1^n)$.

- $k \xleftarrow{\$} \text{Gen}(1^n)$;
- $(m, t) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$;
- Output 1 if $\text{Vrfy}_k(m, t) = 1$ and (m, t) is not a pair seen in the set of queries made by $\mathcal{A}$.

In words, the adversary is able to make queries to get tags for a polynomial amount of messages and then make a message tag pair itself that **can be a message previously queried, but the tag cannot be a tag associated with that message from a previous query.**

Theorem 8.1. *If a MAC with canonical verification is existentially unforgeable under a chosen-message attack, it is also strongly unforgeable under chosen-message attack. (Canonical verification is when you just run the MAC algorithm to see if it matches the tag given.)*

8.1.3 PRFs Make Good MACs for Fixed Length Messages

We construct the following MAC where $F_k : \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a PRF:

$\mathcal{M} = \{0, 1\}^n, \mathcal{K} = \{0, 1\}^n$
Gen : choose $k \leftarrow \mathcal{K}$ uniformly at random
MAC(k, m) : Output (m, t) where $t = F_k(m)$
Ver(k, m, t) : Canonical Verification

Theorem 8.2. *If F_k is a PRF, then the above construction is secure.*

8.1.4 Common Attacks on arbitrary length MACs

Reordering Attack: This attack gets block tags from a query and reorders them. This can be fixed by adding order bits to the message blocks to fix a specific ordering of the blocks for every query.

Length Attack: This attack gets a message with a query and then uses that tag to get the tag for a prefix of the original message. This can be fixed by adding bits encoding the length of the total message to each block.

Mix and Match: This attack queries two or more messages and uses blocks from each of them to get a new message's tag. This can be fixed by adding a random nonce to each block's message. This prevents people from mixing and matching queries as they will likely have different nonces.

8.1.5 MACs for Arbitrary Length Messages

We construct the following MAC where $(\text{Gen}', \text{MAC}', \text{Ver}')$ is secure for n -bit messages:

$\mathcal{M} = \{0, 1\}^{\ell(n)}$, where $\ell(N) = \text{poly}(n)$
Gen : choose $k \leftarrow \text{Gen}'(1^n)$
MAC(k, m) : parse $m = m_1 || m_2 || \dots || m_d$, where $|m_i| = n/4$ pad last blocks with 0s
 let $l = |m|$ and $r \leftarrow \{0, 1\}^{n/4}$, let $t_i \leftarrow \text{MAC}'(k, r || l || i || m_i)$
 output $(r, t_1, \dots, t_d)$
Ver(k, m, t) : Parse m as in **MAC**. Parse $t = (r, t_1, \dots, t_d)$
 Output 1 if $\forall i \in [d], \text{Ver}(k, r || l || i || m_i, t_i) = 1$. Output 0 if not

Theorem 8.3. *If $(\text{Gen}', \text{MAC}', \text{Ver}')$ is secure for n -bit messages, then the above construction is secure.*

8.1.6 Uses

Integrity: Data that has been tampered with will likely have a different tag.

Authentication: Messages have to have originated from a trusted source with access to the private key that can generate valid tags on the messages.

8.2 Quiz Questions

Exercise 24

Which of the following would you choose to construct a message authenticating code (MAC)?

- (a) Encryption Scheme
- (b) Error Correcting Codes
- (c) None of the above

Solution

(c)

Exercise 25

Which of the constructions below is a good idea for MAC for arbitrary-length messages?

- (a) Compute a tag for each block
- (b) Compute a tag for each block along with the block number
- (c) Compute a tag for each block along with the block number and the length of the message
- (d) None of the above

Solution

(d)

Exercise 26

Which of the following can be a secure MAC?

- (a) The tag always contain the first letter of the message
- (b) The tag always starts with the letter "c"
- (c) The key always starts with the letter "c"
- (d) None of the above

Solution

(a),(b),(c)

8.3 Discussion Problems

We will not be testing on CBC mode. So to avoid confusion I will not place the discussion questions here.

9 Hash Functions

9.1 Definitions/Content Review

9.1.1 Collision Resistance

A *hash family* (Gen, H) is a deterministic, efficiently computable function $H : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ for which $|\mathcal{X}| > |\mathcal{Y}|$, equipped with a key-generation algorithm Gen . (Formally, the key space $\mathcal{K}$, domain $\mathcal{X}$, and range $\mathcal{Y}$ can all depend on the security parameter.)

A hash family (Gen, H) is *collision resistant (CR)* if for all PPT attackers $\mathcal{A}$,

$$\text{Adv}_H^{\text{CR}}(\mathcal{A}) \triangleq \Pr_{k \leftarrow \text{Gen}(1^n)} [\mathcal{A}(k) \text{ outputs a collision in } H^k] = \text{negl}(n),$$

where a collision in H^k is a pair of distinct inputs $x \neq x'$ such that $H^k(x) = H^k(x')$. Notice that instead of limiting the attacker to $H^k(\cdot)$ oracle access, we explicitly provide the key k as input. This is to reflect the fact that in real-world applications, in contrast with encryption and authentication keys, hash keys are usually publicly known to all parties (including the attacker). In fact, it is common for standards organizations to choose a single hash key k for the whole world to use. In this case we often speak of $H = H^k$ itself as being collision resistant.

Formally this is not really meaningful, since for any *fixed* hash function there *exists* an efficient algorithm that simply outputs a fixed collision that is “hard-coded” into it; however, such an algorithm may be beyond our present knowledge. The collision-finding game circumvents this issue by requiring the adversary to find a collision for a *randomly chosen* key.

9.1.2 Hash-and-Mac for Arbitrary Length Messages

We construct the following MAC where $(\text{Gen}', \text{MAC}', \text{Ver}')$ is UFCMA-secure for n -bit messages and $H^s(x)$ is a collision resistant hash function from an arbitrary length to n -bits:

$\text{Gen} : \text{choose } k \leftarrow \text{Gen}'(1^n), \text{ choose } s \leftarrow H\text{Gen}(1^n), \text{ Output } (k, s)$
 $\text{MAC}((k, s), m) : \text{Output } \text{MAC}'(H^s(m))$
 $\text{Ver}((k, s), m, t) : \text{Output } \text{Ver}'(k, H^s(m), t)$

This is a lot more efficient than the previous construction.

Theorem 9.1. *If $(\text{Gen}', \text{MAC}', \text{Ver}')$ is secure for n -bit messages and the hash function family $\{H^s\}$ is collision resistant, then the above construction is -secure.*

Proof. Proof sketch. Split into two cases: H has a collision or H does not have a collision. If there is no collision, then MAC security means the probability of finding a correct tag is negligible. The probability of getting a collision must be negligible as well by the Hash security. When you multiply the negligible probability by any advantage, it is still negligible. Thus, the advantage in both cases sum to a negligible probability for the advantage of any adversary.

9.1.3 Merkle Hash Trees

This primitive allows you to store a lot of data in other locations and be confident that it has not been tampered with by just storing the root locally. For more details, see lecture 11.

9.1.4 Uses

Integrity You can tell if data is tampered with just by storing a much shorter hash of the data, as it is unlikely two different sets of data have the same hash.

Arbitrary Length Security A lot of times, you can transfer security from a smaller set to a larger set by hashing the larger inputs.

9.1.5 Big Idea for Proofs

Split the proof into two cases. One case where there is a collision and one case where there is no collision.

9.2 Quiz Questions

Exercise 27

Let H^s be a collision-resistant compression function family. Let s be a random seed generated from $\text{Gen}(1^n)$. Which of the following are correct about H^s ?

- (a) For any $x \neq y$, $H^s(x) \neq H^s(y)$
- (b) For random $x \neq y$, the probability that $H^s(x) = H^s(y)$ is negligible
- (c) For adversarially chosen $x \neq y$, the probability that $H^s(x) = H^s(y)$ is negligible
- (d) None of the above

Solution

(b),(c)

Exercise 28

Which of the following are always true about a Merkle hash tree?

- (a) If the underlying hash function is collision resistant, then it is hard to forge tags
- (b) It supports writing to blocks
- (c) It is time consuming to verify data integrity
- (d) It supports verifying data integrity on reads
- (e) None of the above

Solution

(a),(b),(d)

9.3 Discussion Problems

Exercise 29

A hash family (Gen, H) is *second-preimage resistant (2PR)*, also known as *target-collision resistant (TCR)* if for all PPT attackers $\mathcal{A}$,

$$\text{Adv}_H^{2\text{PR}}(\mathcal{A}) \triangleq \Pr_{\substack{k \leftarrow \text{Gen}(1^n) \\ x \leftarrow \mathcal{X}}} [\mathcal{A}(k, x) \text{ outputs an } x' \neq x \text{ such that } H_k(x') = H_k(x)] = \text{negl}(n).$$

Here the attacker is given a randomly generated hash key k and a random “target” input x , and must find a collision x' with *that specific* x , or in other words, a second preimage of the hash value $H_k(x)$.

Exercise 30

- (a) Why is 2PR no stronger of a security notion than CR?
- (b) Describe a hash function $H: \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$ that is plausibly 2PR, but definitely not CR.

Solution

- | |
|--|
| <p>(a) Intuitively, an attacker that wins the 2PR game also wins the CR game, because it finds a collision (with a specific target value). So, by contrapositive, if a hash family is CR, it must also be 2PR. Hence CR is at least as strong of a security property as 2PR.</p> <p>We can formalize this intuition by a reduction. Suppose (Gen, H) is a collision-resistant hash family. Let $\mathcal{A}$ be any 2PR attacker against H, and build a CR attacker $\mathcal{A}'$ against H</p> |
|--|

as follows:

$\mathcal{A}'(k) : \text{pick a random } x \leftarrow X \text{ and run } \mathcal{A}(k, x) \text{ to get } x' \neq x;$
 output candidate collision (x, x') .

Observe that $\mathcal{A}'$ outputs a collision if and only if $\mathcal{A}$ outputs a second preimage, so $\text{Adv}_H^{2\text{PR}}(\mathcal{A}) = \text{Adv}_H^{\text{CR}}(\mathcal{A}')$. Because (Gen, H) is CR, the right-hand side is $\text{negl}(n)$, so H is also 2PR.

- (b) Let $F : \mathcal{K} \times \mathcal{X} \rightarrow \{0, 1\}^n$ be a collision resistant hash function, where $\mathcal{X} = \{0, 1\}^{2n}$. Define a different hash function $H : \mathcal{K} \times \mathcal{X} \rightarrow \{0, 1\}^n$ as follows:

$$H(x) \triangleq \begin{cases} 0^n & \text{if } x = 0^{2n} \text{ or } x = 1^{2n} \\ F(x) & \text{otherwise} \end{cases}.$$

Clearly, H is not collision resistant because $0^{2n} \neq 1^{2n}$ but $H(0^{2n}) = H(1^{2n})$. We claim, however, that H is 2PR. Let $\mathcal{A}$ be any 2PR attacker against H and build a 2PR attacker $\mathcal{A}'$ against F :

$\mathcal{A}'(k, x) : \text{query } x' = \mathcal{A}(k, x) \text{ and output } x'.$

Since 2PR is no stronger than CR, we have $\text{Adv}_F^{2\text{PR}}(\mathcal{A}') \leq \text{Adv}_F^{\text{CR}}(\mathcal{A}') = \text{negl}(n)$. Thus, the advantage of $\mathcal{A}$ against H is:

$$\begin{aligned} \text{Adv}_H^{2\text{PR}}(\mathcal{A}) &= \Pr_{x \leftarrow \mathcal{X}}[x \in \{0^{2n}, 1^{2n}\} \text{ and } H(x) = H(x')] + \Pr_{x \leftarrow \mathcal{X}}[x \notin \{0^{2n}, 1^{2n}\} \text{ and } H(x) = H(x')] \\ &\leq \Pr_{x \leftarrow \mathcal{X}}[x \in \{0^{2n}, 1^{2n}\}] + \Pr_{x \leftarrow \mathcal{X}}[x \notin \{0^{2n}, 1^{2n}\} \text{ and } F(x) = F(x')] \\ &\leq \frac{2}{2^{2n}} + \text{Adv}_F^{2\text{PR}}(\mathcal{A}') = \text{negl}(n) + \text{negl}(n) = \text{negl}(n). \end{aligned}$$

Therefore, H is 2PR but not CR.

10 Number Theory

10.1 Definitions/Content Review

10.1.1 Groups

A **group** $(\mathbb{G}, \times)$ is a set, $\mathbb{G}$, and a binary operation, $\times$ such that

1. There exists an identity element, 1 , such that $1 \times g = g, \forall g \in \mathbb{G}$
2. $\mathbb{G}$ is closed under the operation. That is $\forall g_1, g_2 \in \mathbb{G}, g_1 \times g_2 \in \mathbb{G}$
3. All elements have an inverse. That is $\forall g_1 \in \mathbb{G}, \exists g_1^{-1} \in \mathbb{G}$, st $g_1 \times g_1^{-1} = 1$
4. The operation is associative. That is $\forall g_1, g_2, g_3 \in \mathbb{G}, (g_1 \times g_2) \times g_3 = g_1 \times (g_2 \times g_3)$

An **abelian group** is a group where the operation is commutative. That is $\forall g_1, g_2 \in \mathbb{G}, g_1 \times g_2 = g_2 \times g_1$

A **subgroup** is a subset within a group that is also a group with the same operation.

A **cyclic group** is a subgroup of a finite order (size) group that is generated by an element g . That is we say $\langle g \rangle$ is a cyclic group generated by g if $\langle g \rangle = \{g^0, g^1, \dots\}$

$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ for prime p is a group with order $p-1$.

The order of an element g in a cyclic group is the smallest integer i such that $g^i = 1$

10.1.2 Potentially Useful Theorems

Theorem 10.1. Let $\mathbb{G}$ be a finite group. Let g be an element of order i . Then for any x , $g^x = g^{x \bmod i}$

Theorem 10.2. If $\mathbb{G}$ is a group with prime order p , then $\mathbb{G}$ is cyclic. Furthermore, all elements in the group except 1 can be the generator.

10.2 Quiz Questions

Exercise 31

Consider group $G = \mathbb{Z}_5^*$ with respect to $\times \bmod 5$. What's the order of $g = 2$?

- (a) 1
- (b) 2
- (c) 3
- (d) 4
- (e) None of the Above

Solution

(d)

Exercise 32

Consider group $G = \mathbb{Z}_5^*$ with respect to $\times \bmod 5$. Which of the following are true?

- (a) $g^7 = g^4$
- (b) $g^9 = g^5$
- (c) G is a cyclic group with generator g

- (d) G is not cyclic
- (e) None of the Above

Solution

(b),(c)

10.3 Discussion Problems

We will not be test on the discussion problems related to this topic. Just know that there exists an algorithm (extended eulclidean algorithm) that can find inverses in $\mathbb{Z}_p^*$ and that there is a way to sample primes randomly (with overwhelming probability) because there is an algorithm that can test the primality of a number (with overwhelming probability).

11 Key Exchange and Number Theory Assumptions

11.1 Definitions/Content Review

11.1.1 Discrete Log Assumption

Consider the following experiment

Experiment $\text{DLOG}^{\mathcal{A}, \mathcal{G}}(1^n)$.

- Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$, where $\mathbb{G}$ is a cyclic group of prime order q and g is a generator of $\mathbb{G}$.
- Let $h \xleftarrow{\$} \mathbb{G}$
- Let $x \leftarrow \mathcal{A}(\mathbb{G}, q, g, h)$
- Output 1 if $g^x = h$.

We say that a p.p.t. adversary $\mathcal{A}$ breaks the discrete logarithm problem if

$$\Pr[\text{DLOG}^{\mathcal{G}, \mathcal{A}}(1^n) = 1] \geq \frac{1}{p(n)}.$$

for some polynomial $p(\cdot)$.

11.1.2 Decisional Diffie-Hellman Assumption

Consider the following experiment

Experiment $\text{DDH}_b^{\mathcal{A}, \mathbb{G}}(1^n)$.

- Run $\mathcal{G}(1^n)$ to obtain $(\mathbb{G}, q, g)$, where $\mathbb{G}$ is a cyclic group of prime order q and g is a generator of $\mathbb{G}$.
- Let $x, y, z \xleftarrow{\$} \mathbb{Z}_q$
- If $b = 0$, $\mathcal{A}$ is given $(\mathbb{G}, q, g, g^x, g^y, g^z)$, else $\mathcal{A}$ is given $(\mathbb{G}, q, g, g^x, g^y, g^{xy})$
- Output $\mathcal{A}$'s output.

We say that a p.p.t. adversary $\mathcal{A}$ breaks the DDH problem if

$$|\Pr[\text{DDH}_0^{\mathcal{G}, \mathcal{A}}(1^n) = 1] - \Pr[\text{DDH}_1^{\mathcal{G}, \mathcal{A}}(1^n) = 1]| \geq \frac{1}{p(n)}.$$

11.1.3 Key Exchange Security

Consider the following experiment

Experiment $\text{KEEXPT}_b(1^n)$.

- Two parties with input 1^n run the key exchange protocol. Let Γ be the transcript of the protocol and k the outputted key.
- If $b = 0$, output (Γ, k) . Else, output (Γ, k') where $k' \leftarrow \mathcal{K}$

We say that the key exchange is secure if for all p.p.t. adversaries $\mathcal{A}$

$$\text{KEEXPT}_0(1^n) \approx \text{KEEXPT}_1(1^n)$$

Theorem 11.1. *If DDH is true with respect to $\mathbb{G}$, then discrete log is true with respect to $\mathbb{G}$*

11.1.4 Diffie-Hellman Key Exchange

Here is the Diffie-Hellman Protocol:

1. Alice samples a prime-order multiplicative group $(\mathbb{G}, q, g)$ where $q = |\mathbb{G}|$ and g is the generator. Alice sends $(\mathbb{G}, q, g)$ to Bob.
2. Alice picks a secret $x \in \mathbb{Z}_q$ that she does not reveal. She then computes $A = g^x$ and sends to Bob.
3. Bob picks $y \in \mathbb{Z}$ that he keeps secret. He then computes $B = g^y$ and sends to Alice.
4. Alice and Bob use their secret integers to compute $k_A = A^y$ and $k_B = B^x$.

Theorem 11.2. *If DDH is hard with respect to $\mathcal{G}$, then Diffie-Hellman Key Exchange is secure*

11.1.5 Uses

Diffie-Hellman Key Exchange is useful for securely sending secret keys that people can agree on for secret key cryptography we have discussed in class before. However, Diffie-Hellman Key Exchange breaks when considering active adversaries.

11.2 Quiz Questions

Exercise 33

Which of the following statements must be true

- (a) DDH assumption is at least as strong as the Discrete log assumption
- (b) Discrete log assumption is at least as strong as the DDH assumption
- (c) DDH assumption is the same strength as the Discrete log assumption

Solution

(a)

12 Public-Key Cryptography

12.1 Definitions/Content Review

12.1.1 Correctness

Except with negligible probability over the choice of sk, pk output by $\text{Gen}(1^n)$, it must be that for any legal message:

$$\text{Dec}(sk, \text{Enc}(pk, m)) = m$$

Note that this is not 100 percent certainty because there is no way to sample primes with 100 percent certainty.

12.1.2 Security

Consider the following experiment

Experiment $\text{PKEXPT}_b^{\mathcal{A}}(1^n)$.

- $sk, pk \xleftarrow{\$} \text{Gen}(1^n)$;
- $(m_0, m_1) \leftarrow \mathcal{A}(1^n, pk)$;
- $c \leftarrow \text{Enc}(pk, m)$;
- $b' \leftarrow \mathcal{A}(c)$.

- Output b' .

We say that the key exchange is EAV-secure if for all p.p.t. adversaries $\mathcal{A}$ there exists some negligible function, $\text{negl}(\cdot)$, such that for all n :

$$\Pr[\text{PKEXPT}_1^{\mathcal{A}}(1^n) = 0] - \Pr[\text{PKEXPT}_0^{\mathcal{A}}(1^n) = 0] \leq \text{negl}(n)$$

Theorem 12.1. *If a public-key encryption is EAV-secure, then it is CPA-Secure. This implies public-key encryption must be randomized.*

12.1.3 El Gamal Encryption

Run $\mathcal{G}(1^n)$ and get a group description $(\mathbb{G}, q, g)$, where $g \in \mathbb{G}$ is a generator and $q = |\mathbb{G}|$ is the group order, the scheme is defined as follows (the message space is the group $\mathbb{G}$):

Gen : choose random $x \leftarrow \mathbb{Z}_q$ and output $pk = (\mathbb{G}, q, g, h = g^x), sk = (\mathbb{G}, q, g, x)$

Enc($pk, m \in \mathbb{G}$) : choose random $y \leftarrow \mathbb{Z}_q$ and output $(g^y, \underbrace{m \cdot g^{xy}}_{m \cdot h^y})$

Dec($sk, (c_1, c_2)$) : output $c_2 \cdot (c_1^x)^{-1} \rightsquigarrow c_2 \cdot (c_1^x)^{-1} = m \cdot g^{xy} \cdot g^{-xy} = m$.

Theorem 12.2. *El Gamal encryption is CPA-Secure if DDH is hard with respect to $\mathcal{G}$*

12.1.4 Uses

Public key cryptography allows people to encrypt messages to you without communicating an agreed upon key that needs to be kept secret from adversaries. However, these operations in practice are a bit more expensive than secret key cryptography.

12.2 Quiz Questions

Exercise 34

Which of the following are true?

- (a) Public-key encryption encrypts using the secret key and decrypts with the public key
- (b) Public-key encryption encrypts using the public key and decrypts with the secret key
- (c) If a public key encryption scheme is EAV-secure, then it is CPA secure
- (d) None of the above

Solution

(b),(c)

Exercise 35

Which of the following are true?

- (a) A deterministic public-key encryption scheme can be EAV-secure
- (b) A randomized public-key encryption scheme can be perfectly secure
- (c) A deterministic public-key encryption scheme cannot be EAV-secure
- (d) There is no perfectly secure public-key encryption scheme
- (e) None of the above

Solution

(c),(d)

12.3 Discussion Problems

Exercise 36

Let us consider an ElGamal cryptosystem, with publicly known parameters $p = 43$ and $g = 5$. Encrypt the message $m = 22$ for Alice, whose public key is 11. Use the random $b = 29$ in the process. Verify the result by decrypting the ciphertext with Alice's private key 18. You can check this with a calculator.

Solution

$(C = 22 \cdot 11^{29}, B = 5^{29})$ $(B^{18})^{-1} \cdot 22 \cdot 11^{29} = 22$
